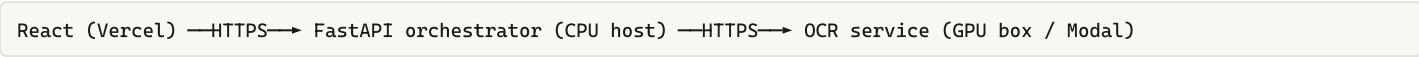


Hermes — Technical Spec

Stack: FastAPI (backend) · React + Vite + TypeScript (frontend) · Unlimited-OCR on GPU · Claude `cclaude-opus-4-8` · Python 3.13. Backend holds the Claude key and the audit engine; the OCR model is the only GPU piece, behind one HTTP contract (swap GPU box ↔ Modal by changing a URL).

Data flow: upload → PDF→images (PyMuPDF) → OCR service (markdown + grounding boxes) → Claude extraction (typed schema) → hermes rule engine (deterministic audit) → result stored → frontend polls.



Repo layout

- `hermes/` — verification engine. `core.py` = `Finding`, `Severity`, `run_pack`, `render_report`; `financial.py` = schema + rules; `demo_financial.py` = runnable demo.
- `apps/api/` — FastAPI orchestrator. `main.py` (routes), `pipeline.py` (orchestration), `extraction.py` (Claude), `ocr_client.py` (OCR seam), `jobs.py` (in-mem store), `ratelimit.py` (cost guard), `config.py`, `smoketest.py`.
- `apps/web/` — React + Vite frontend **[partner]**.
- `services/ocr/` — Unlimited-OCR HTTP service, uvicorn on the box + a Modal entry, same handler **[todo]**.

Backend API (uvicorn `apps.api.main:app --reload, base http://localhost:8000`)

- GET `/health` → { `ok`, `mock_ocr`, `mock_extraction`, `model` }
 - POST `/documents` — multipart field `file` → { `job_id`, `status` }. Errors: 400 empty, 429 rate-limited (detail).
 - GET `/documents/{job_id}` → { `job_id`, `status`, `filename`, `result`, `error` }. `status` ∈ `queued` | `processing` | `done` | `error`; `result` is null until `done`.
- result (when done):** { `filename`, `pages`, `markdown`, `entity`, `period`, `currency`, `summary`:{`checks`,`passed`,`flagged`}, `findings`: [...], `report_text` } where each finding = { `rule`, `passed`, `severity`(`error`|`warning`|`info`), `message`, `expected`, `actual`, `citations`:`[str]` }.
- Async model: POST creates a job + a `BackgroundTask`; client polls GET. Job store is in-memory for now → swap for Postgres when persistence is needed.

OCR service contract (`services/ocr`)

POST `/ocr` { `filename`, `grounding:bool`, `file_b64` } → { `markdown`, `boxes`:[{`text`,`bbox`,`page`}], `pages`, `latency_ms` }.

Grounding: prompt prefix `<grounding>` makes the model emit `<det>``<span [x1,y1,x2,y2]` boxes (0–1000 scale; rescale to pixels). The same handler runs on the GPU box (uvicorn, exposed via Cloudflare Tunnel/Tailscale) or on Modal serverless — the orchestrator only sees this URL.

Verification engine (`hermes`)

- A **rule** = `fn(document) → list[Finding]`; a **pack** = an ordered list of rules. Drive with `run_pack(pack, doc)`, format with `render_report(findings)`.
- Rules are **pure arithmetic/logic — no LLM** → deterministic and auditable. The LLM only extracts + explains; the verdict is code.
- Add a domain pack = new Pydantic schema + a new rule list. No engine changes.
- `FINANCIAL_RULE_PACK`: `gross_profit`, `operating_income`, `net_income`, `balance_sheet_identity`, `crossfoot_{assets,liabilities,equity}`, `bank_reconciliation`, `duplicate_line_item`, `benford`.
- Demo: `python -m hermes.demo_financial` → 10 checks, 4 flagged on a planted-error sample.

Config (env vars) + mock modes

var	default	note
<code>ANTHROPIC_API_KEY</code>	—	backend only; unset → <code>mock_extraction</code> (returns sample statement)
<code>OCR_SERVICE_URL</code>	—	unset → <code>mock_ocr</code> (returns fixture markdown)
<code>CLAUDE_MODEL</code>	<code>cclaude-opus-4-8</code>	
<code>CORS_ORIGINS</code>	<code>*</code>	comma-separated for prod (the Vercel origin)
<code>RATE_LIMIT_PER_MIN</code> / <code>DAILY_JOB_CAP</code>	<code>5 / 200</code>	open-access cost guard

Mock modes auto-enable when their dependency is unset, so the **entire backend runs with no GPU and no API key**.

Run / verify (Windows, global installs)

```
pip install fastapi "uvicorn[standard]" python-multipart httpx anthropic pymupdf
python -m hermes.demo_financial           # engine only
python -m apps.api.smoketest               # full flow in mock mode → asserts 4 flagged  [PASSES]
uvicorn apps.api.main:app --reload         # serve the API
```

Frontend integration (`apps/web`) **[partner]**

React + Vite + TS · Tailwind/shadcn · TanStack Query. `VITE_API_URL` env points at the backend. Poll GET `/documents/{id}` every ~1.5 s until `done/error`; show a "warming up" state on OCR cold-start. Screens: **Upload** → **Processing** → **Results** (left = rendered markdown;

right = findings, flagged first, colored by severity, each with citations; summary counts + download report_text). Handle 429 by surfacing detail.

Deploy

web → Vercel · api → Render/Railway/Fly (CPU, HTTPS) · ocr → Modal serverless (or the GPU box via tunnel). Secrets server-side only.